Memory Allocation Using Segmentation

May 10, 2026

Brief Explanation

This project demonstrates memory allocation using segmentation with two strategies: First-Fit and Best-Fit. The GUI shows the memory layout, the segment table, and the free holes after each operation.

Simple Code Explanation

The code is split into a few main parts:

- **MemoryManager**: this is the part that does the real work. It keeps track of free holes and decides where each process will go.
- **Process, Segment, and Hole**: these are small data models. They store the name, size, and address of each process part or free space.
- **MainWindow**: this controls the GUI. It connects the buttons, tables, and memory view to the allocation logic.
- **MemoryCanvas**: this draws the memory blocks on the screen so the user can see what is allocated and what is free.
- **main_window.ui and styles.qss**: these files define the interface layout and the visual style.

In short, the program starts with free holes, then it places each process into memory using First-Fit or Best-Fit. After each step, the GUI updates the tables and the memory drawing so the result is easy to follow.

Important Code Snippets

1. Memory allocation

This part tries to place every segment of a process into memory.

```
def allocate_process(self, process, method):
    working_holes = [Hole(hole.start, hole.size) for hole in self.holes]
    for segment in process.segments:
        hole_index = self._find_hole_index(working_holes, segment.size, method)
        if hole_index is None:
            process.allocated = False
            return False
        hole = working_holes[hole_index]
        segment.start = hole.start
        hole.start += segment.size
        hole.size -= segment.size
    self.holes = sorted(working_holes, key=lambda hole: hole.start)
    process.allocated = True
    return True
```

2. Choosing the hole

This part decides where to put the next segment.

```
def _find_hole_index(holes, size, method):
    if method == AllocationMethod.FIRST_FIT:
        for index, hole in enumerate(holes):
            if hole.size >= size:
                return index
        return None

    best_index = None
    best_waste = None
    for index, hole in enumerate(holes):
        if hole.size >= size:
            waste = hole.size - size
            if best_waste is None or waste < best_waste:
                best_index = index
                best_waste = waste
    return best_index
```

3. Running the test case

This part runs the same steps shown in the screenshots.

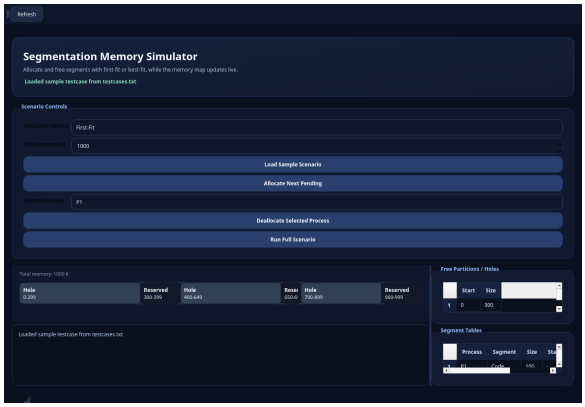
```
steps = [
    ("allocate", "P1"),
    ("allocate", "P2"),
    ("allocate", "P3"),
    ("deallocate", "P1"),
    ("allocate", "P4"),
]
```

Test Case

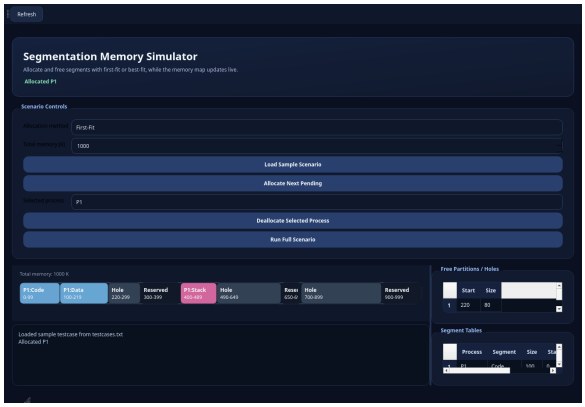
- Total memory: 1000 K
- Initial holes: $[0, 300]$, $[400, 650]$, $[700, 900]$
- Operations: allocate P1, allocate P2, allocate P3, deallocate P1, allocate P4
- The scenario is run once with First-Fit and once with Best-Fit

Screenshots

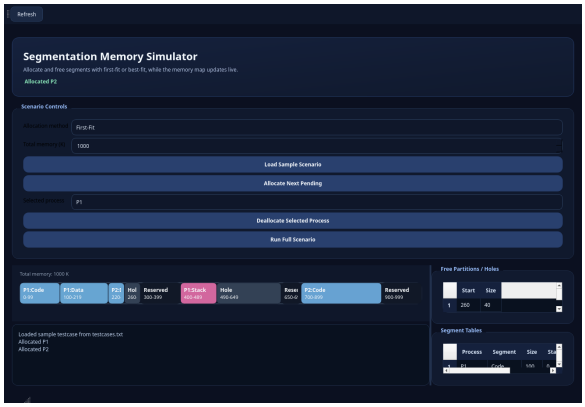
First-Fit



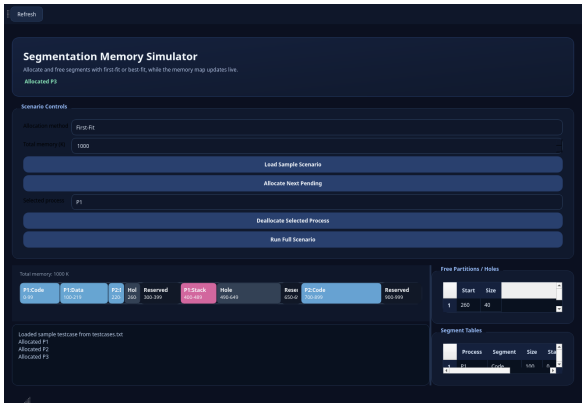
(a) Initial



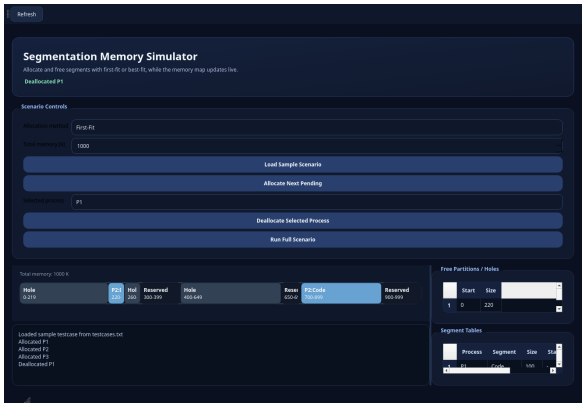
(b) P1 allocated



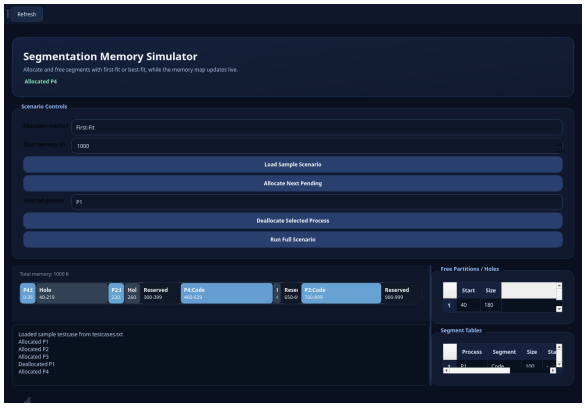
(c) P2 allocated



(d) P3 attempted



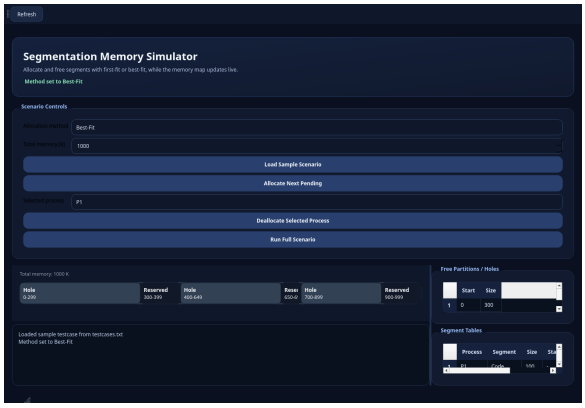
(e) P1 deallocated



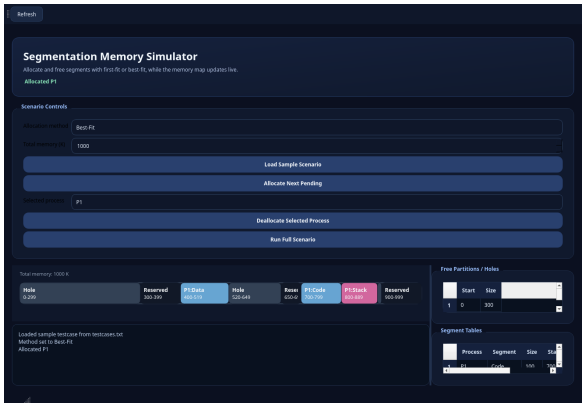
(f) P4 allocated

Figure 1: First-Fit screenshots after each operation.

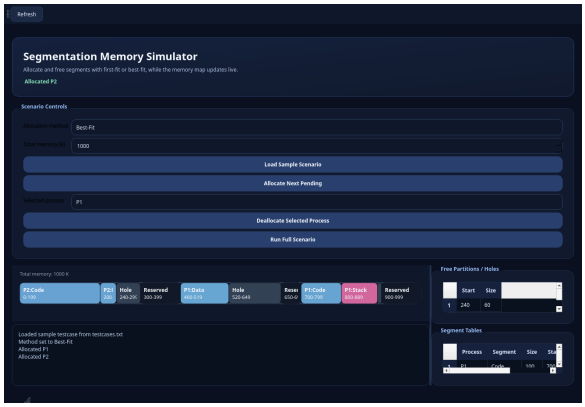
Best-Fit



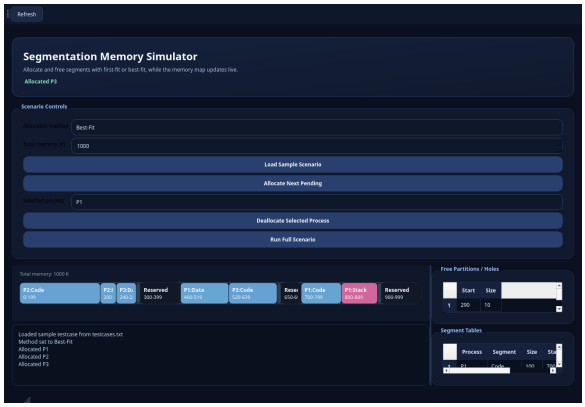
(a) Initial



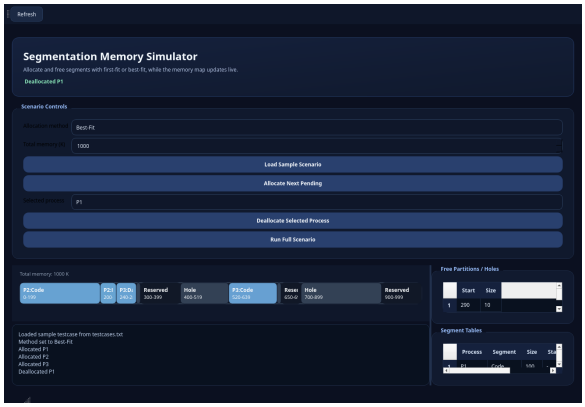
(b) P1 allocated



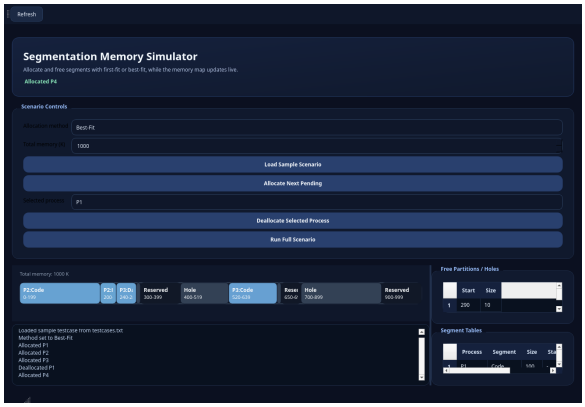
(c) P2 allocated



(d) P3 attempted



(e) P1 deallocated



(f) P4 allocated

Figure 2: Best-Fit screenshots after each operation.

Short Comparison

First-Fit chooses the first hole that fits, so it is faster. Best-Fit searches for the smallest suitable hole, so it usually reduces wasted space. The screenshots show how the two methods place the same processes differently.

Conclusion

The project successfully shows segmentation-based allocation with a simple GUI and screenshot evidence for both algorithms.